MCA

Abhay Kiran
2611006200002

1) Diff b/w von neuman and harvard architecture

Von Neuman Architecture :-

→ Hugh level of flexibility as memory is shared between instructions and data so that level assigned to each can fluctuate based on task
→ Speed is limited when compared to harvard due to only having "1" memory location and set of buses
→ Used in general purpose computer that will be used for many different purposes.

Harvard Architecture

→ Limited flexibility as there is only a certain amount of memory that can be used for data and a certain amount for instructions
→ Two set of memory and buses mean data can be handled more quickly which decreases execution time
→ Typical in embedded system like washing machines.

2) What is Endianess? Explain type of Endianess with example.

→ Order in which a computer stores multi byte numbers in memory. It decides whether the system writes the "big end" MSB first or "little end" into 1st memory slot.

1) Big Endian :-

MSB stored at lowest memory address :- Eg 0x12345678

0x1000 → 0x12 → MSB
0x1001 → 0x34
⋮
0x1004 → 0x78 → LSB

2) Little Endian :-

LSB stored at lowest memory address

0x1000 → 0x78 → LSB
⋮
0x1004 → 0x12 → MSB

2) Features of LPC 21xx and LPC 23xx

[ARM-7]
↓
[AHB/APB]

[Flash] [SRAM] [GPIO] [Timer] [UART] [SPI/I²C] [ADC] [PWM]

[watchdog]
↓
[CAN] → LPC 23xx only

LPC 21xx Features

→ ARM 7 32 bit RISC processor
→ 60 MHz clock
→ Single High performance bus
→ No DMA controller
→ 40 kB SRAM
→ No ethernet
→ USB 2.0
→ No storage expansion

LPC 23xx Features

→ ARM 7 TDMI
→ 72 MHz clock
→ Dual layer AHB bus matrix
→ General purpose and dedicated
→ 96 kB SRAM (Split SRAM)
→ On chip Ethernet dedicated DMA
→ USB 2.0, Host and OTG
→ SD/MMC card Interface

3) Explain GPIO program

[GPIO port] → [GPIO Register] → [IODIR, IOSET, IOCLR] → [GPIO Pin] → [LED]

```
#include <LPC21xx.h>
int main (void) {
IODIR0 |= (1<<10);
  while (1)
  {
    IOSET0 |= (1<<10); //LED ON
     delay (1000)
    IOCLR0 &= ~(1<<10); //LED off
     delay (1000);
```

IODIR → configures port/pin as I/O
IOSET → set as logic high
IOCLR → set to low
IOPIN → current status.

) Explain LED and interfacing with ARM

→ LED connected to GPIO Pin through a resistor
→ Configure the GPIO pin as O/P using IODIR
→ Make pin high using IOSET
→ Make pin low using IOCLR
Program same as previous.

) Explain SFR involved in Timer module.

TCR → Timer Control Register → Enables disables or resets the timer.
TC → Timer Counter → Holds the current count value
PR → Pre Scale register → Divides I/P to slow down timer
MR → Match Register → Stores target value
IR → Interrupt register → Indicates pending timer interrupts and clears
MCR → Match control → Specifies action on a match
CR → Capture Register → Stores the captured timer count
CR → Capture of timer value at signal high.

) Explain working of timer and programming

→ Peripheral clock is supplied to timer
→ Pre scaler register divides the clock frequencies
→ Timer counter increments after prescaler counts its value
→ When TC equals match register (MR) value a match event occurs
→ Depending on MCR timer can interrupt

Program :

```
#include <LPC21xx.h>
void delay(void){
  T0TCR = 0x02      → Resets timer 0
  T0PR  = 5aaaa     → 60MHz → 1mS
  T0MR0 = 1000      → match after 1000 counts = 1 second
  T0MCR = 0x04      → Stop timer on match
  T0TCR = 0x01      → Start timer
  while(!(T0IR & 0x01))
    T0IR = 0x01     → Interrupt flag
```

a) 10ms delay on Pin 0.4

```
#include <LPC21xx.h>
void delay_10ms (void){
int main ()
{
  IODIR0 = (1<<4);
while (1) {
   IOSET = (1<<4);
   delay_10ms ();
   IOCLR0 = (1<<4);
   delay_10ms ();
  }
}
```

```
void delay_10ms (void){
 TOTCR = 0x02;
 TOPR = 5aaaa;
 TOMR0 = 10;
 TOMCR = 0x04;
 TOTCR = 0x01
while (!(TOIR &0x01));
   TOIR = 0x01;
 }
```

b) 10ms delay Pin 0.6

```
#include <LPC21xx.h>
 void delay_10ms (void);
  int main () {
  IODIR |= (1<<4);
while (1){
 IOSETO = (1<<4);
  delay_10μs();
 IOCLR = (1<<4);
 delay_10ms();
  }
}
```

```
void delay_10us(void)
{
  TOTCR = 0x02
   TOPR = 5a;
   TOMR0 = 10;
   TOMIR = 0x04
   TOTCR = 0x01;
 while (!TOIR = 0x01);
   TOIR = 0x01;
}
```

c) Explain SFR involved in PLL module of ARM in C

PLL0CON → PLL control register → Enables or disconnects PLL

PLLCFG → PLL configuration register → Sets the multiplier, divider and FP to obtain frequency required

PLL STAT → PLL status register - Displays current PLL status, lock status and CFG values

PLLFEED → PLL Feed Register → Used to update PLL setting

0xAA & 0x55.

10) Explain PLL module of ARM microcontroller

→ External crystal provides the input clock

→ Configure desired multiplier and divider using PLLCFG

→ Enable PLL using PLLCON

→ Load feed sequence using 0xAA and 0x55 to PLLFEED

→ Wait till lock bit in PLLSTAT is set.

→ Connect the PLL using PLLCON repeat feed sequence

→ CPU now runs at higher clock frequency.

11) Explain steps in programming PLL of ARM μC.

```
#include <LPC21xx.h>
void PLL_init (void)
{
  PLLCFG = 0x24      => M=5 & P=2
  PLLCON = 0x01      -> Enable PLL

  PLLFEED = 0xAA;
  PLLFEED = 0x55;    ] Feed sequence
  while(!(PLLSTAT & (1<<10)));  -> wait till PLL locks

    PLLCON  = 0x03;
    PLLFEED = 0xAA
    PLLFEED = 0x55;
}
 int main ();
{
    PLLInit();
  while()
   {
   }
}
```